



University of
Nottingham

UK | CHINA | MALAYSIA

Software Engineering

Object Oriented Design & Test Planning

COMP1003 - Lecture 9

Dr Horia A. Maior



- Meme of the week
- Feedback from Lab 3
- Lecture



University of
Nottingham

UK | CHINA | MALAYSIA

Meme of the Week



University of
Nottingham

UK | CHINA | MALAYSIA

Feedback from Lab 3



Good diagrams

- Clear Reasoning included in .md file
- Links to Previous Requirements
- Clarity and Readability
- Logical Flow and Accuracy
- Appropriate Level of Detail



University of
Nottingham

UK | CHINA | MALAYSIA

Software Engineering

Object Oriented Design & Test Planning

COMP1003 - Lecture 9

Dr. Horia A. Maier

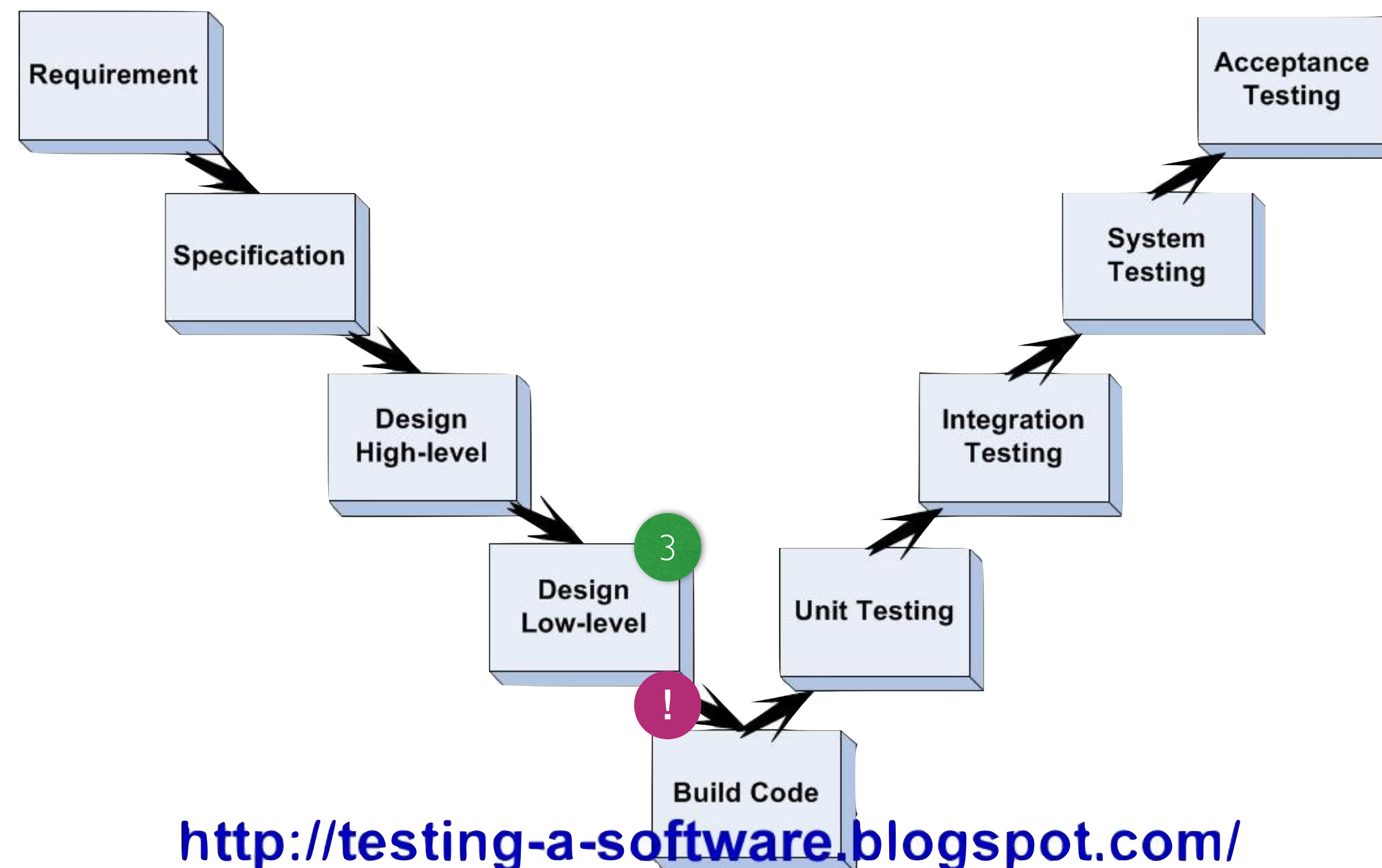


Today's Learning Outcomes

- 0) Low-level designs are there to guide developers
- 1) These are the final outputs of the Spec phase
- 2) Planning testing is part specifying phase

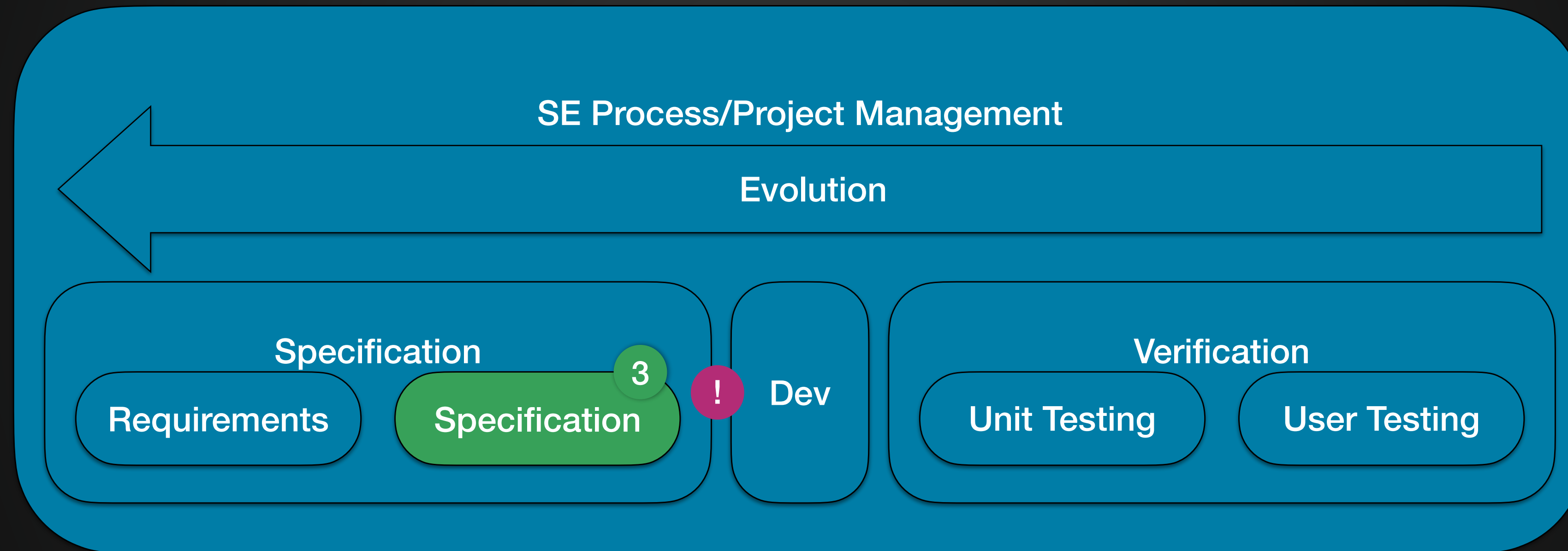


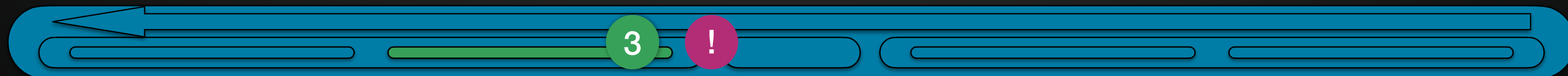
Keeping Track of This Module





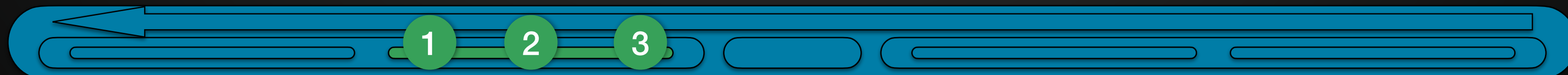
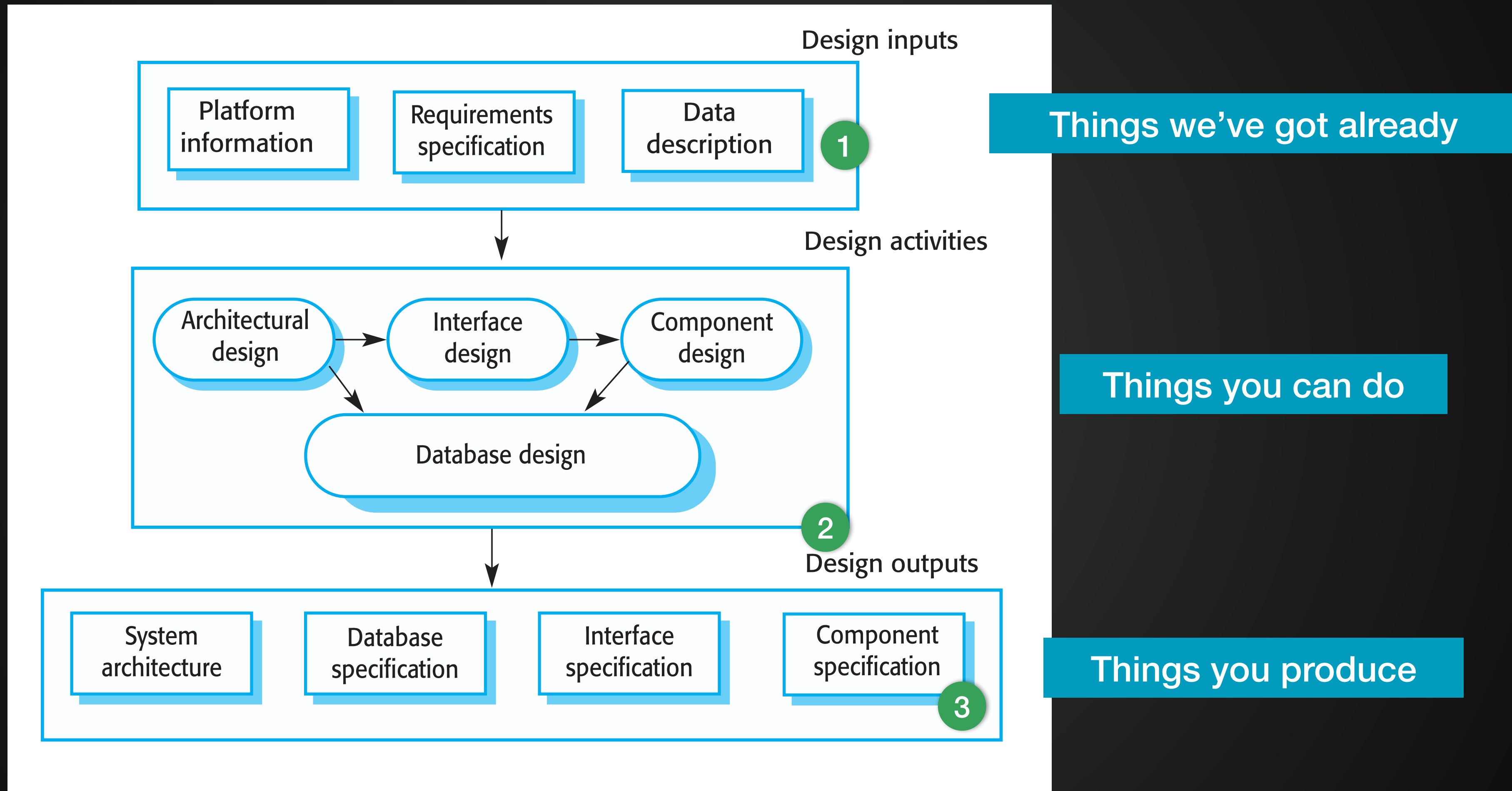
Keeping Track of This Module







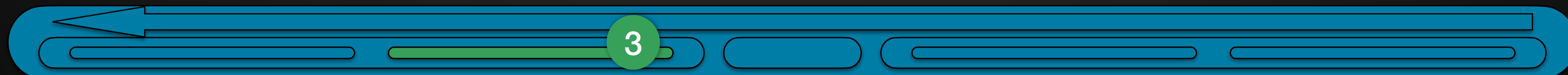
Where we are ...





What you need for Low-Level Design?

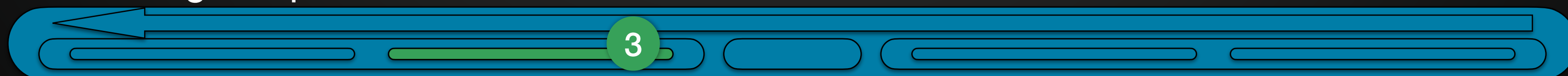
- We want to have enough detail to give to the development team
- “Here, you build this”
 - Not - here, please design this software as go
- Creating a final low-level design is often led by a Solution Architect
 - “Perhaps the most important tool in the toolbox is a visual documentation language, such as UML.”
 - “In addition to UML, the SA may need to be good at database design.”
 - “Perhaps the most critical skills for the SA are the ability to create consensus and understanding around the architecture.”
- Its often the final version of all the design documents, read by the subsequent teams.





Some notes on final Spec outputs

- **Architecture Design**
 - Now we are talking about the internal architecture of the objects, classes, use of APIs
 - Like MVC architecture (not covered in this module)
- **Interface Design**
 - Detailed 'blueprints' of final user interface (not just drawings, but specifics - e.g. form validation too)
 - Not just the 'user' interface, also any interfaces with other services
 - Or between components (that might be developed by different teams)
 - the web team
 - the mobile dev team
 - the server-side team
- **Database Design**
 - Developers will presume that they can request from the DB in their code, as specified
 - Other types of db, however, could be specified
- **Component Design**
 - The exact class structure inside each component
 - Or indeed the logic captured in haskell-built server side code



Component Design in UML

- Presuming Object-Oriented (in this module)
 - Classes that represent objects
 - Objects store data
 - Objects have responsibilities
- The missing element of Textual Analysis so far

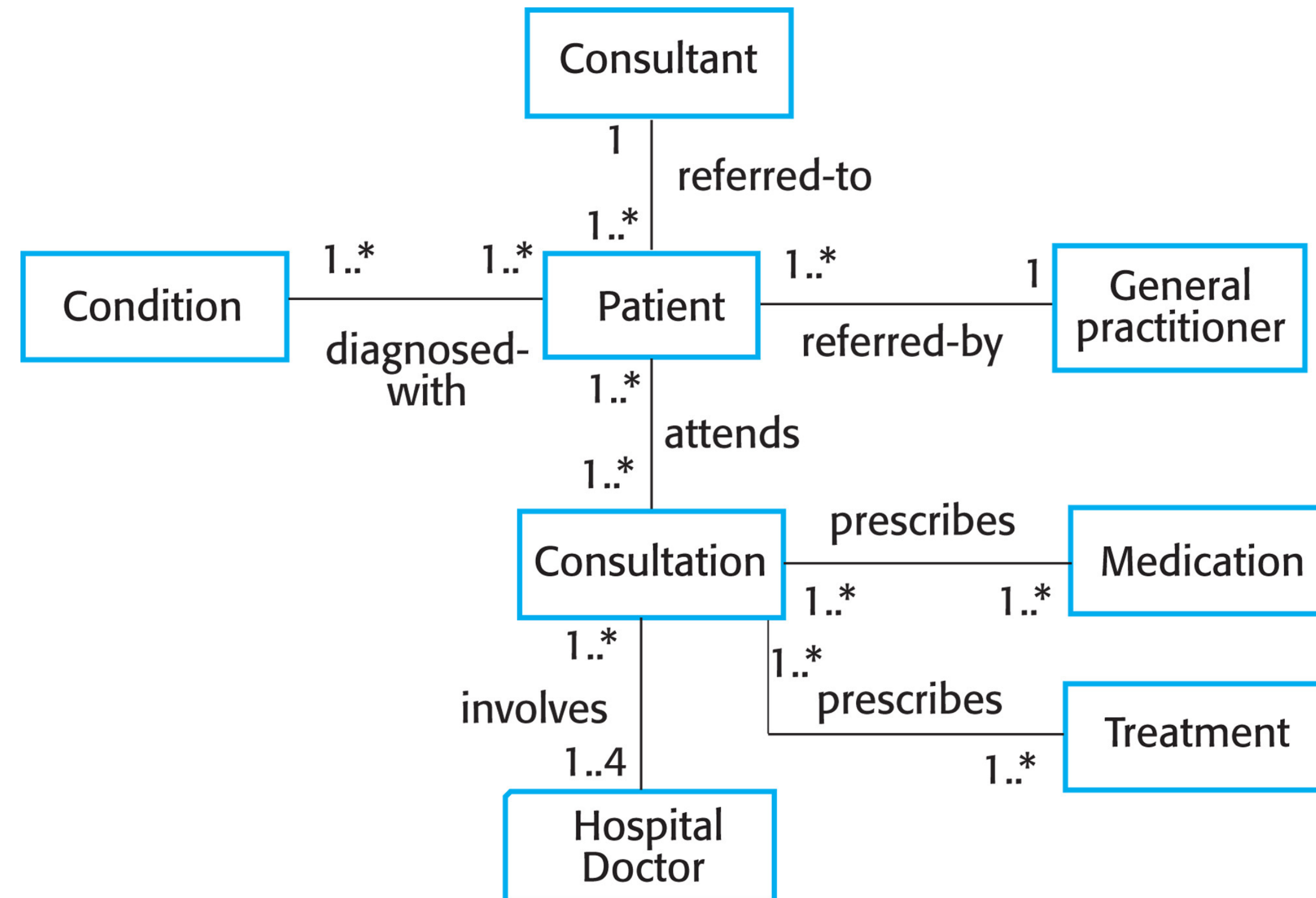
The screenshot displays a software interface for UML modeling. On the left, a table lists candidate classes extracted from text. The table has four columns: No., Candidate Class, Extracted Text, and Type. The third row, 'live programs', is highlighted. To the right of the table, a dropdown menu is open, showing various UML element types. The 'Actor' option is currently selected and highlighted in orange. In the background, a document with text is visible, containing phrases like 'delivers both paid and free online television broadcasting services to all TV fans' and 'Members can register as a premium member to watch both archived and live programs for US \$30 per'.

No.	Candidate Class	Extracted Text	Type
1	general member	general member	Actor
2	premium member	premium member	Actor
3	live programs	live programs	Use Case
4	archived TV program	archived TV program	Unspecified

Dropdown menu options:

- Class
- Actor
- Use Case
- Requirement
- Action
- Candidate Item...

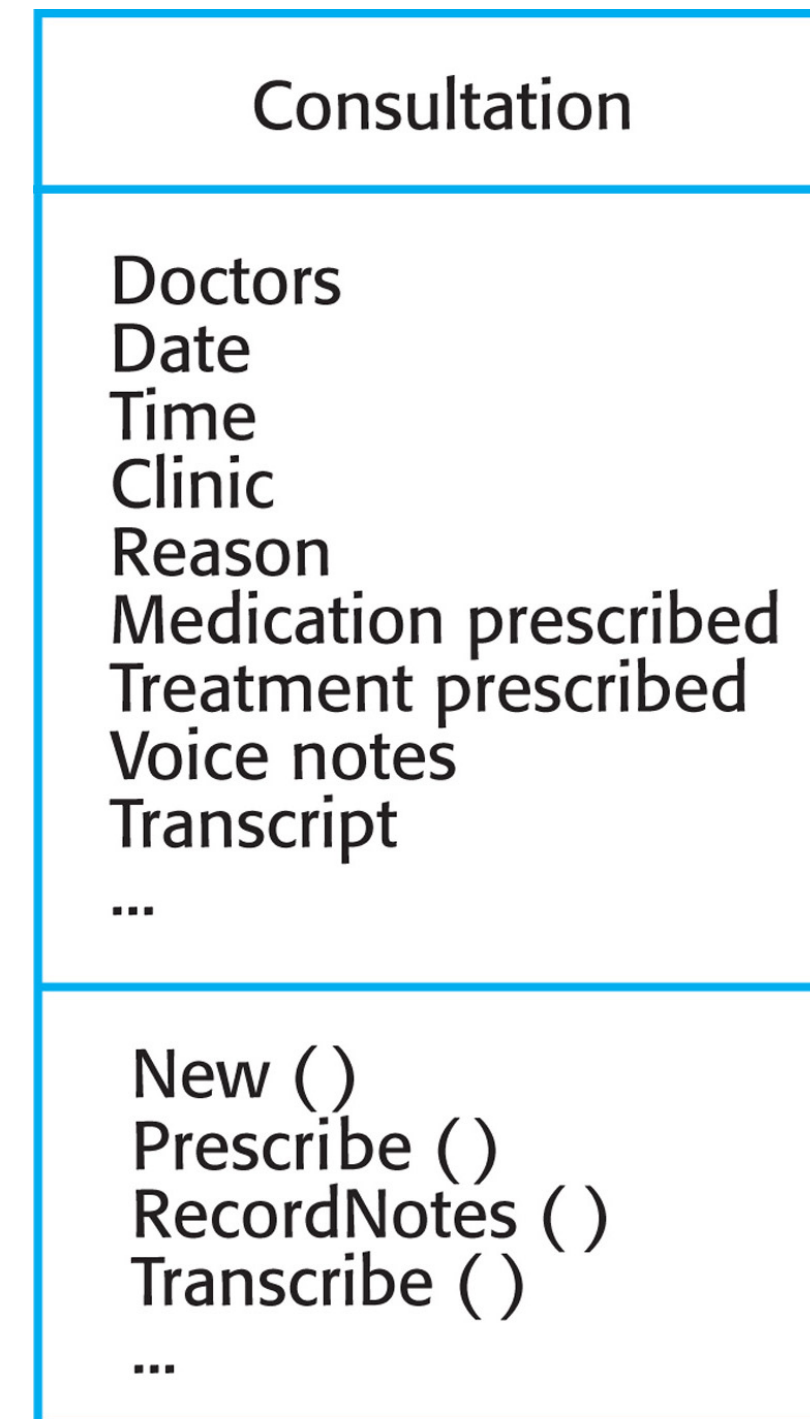
Figure 5.9 Classes and associations in the Mentcare system



Copyright ©2016 Pearson Education, All Rights Reserved



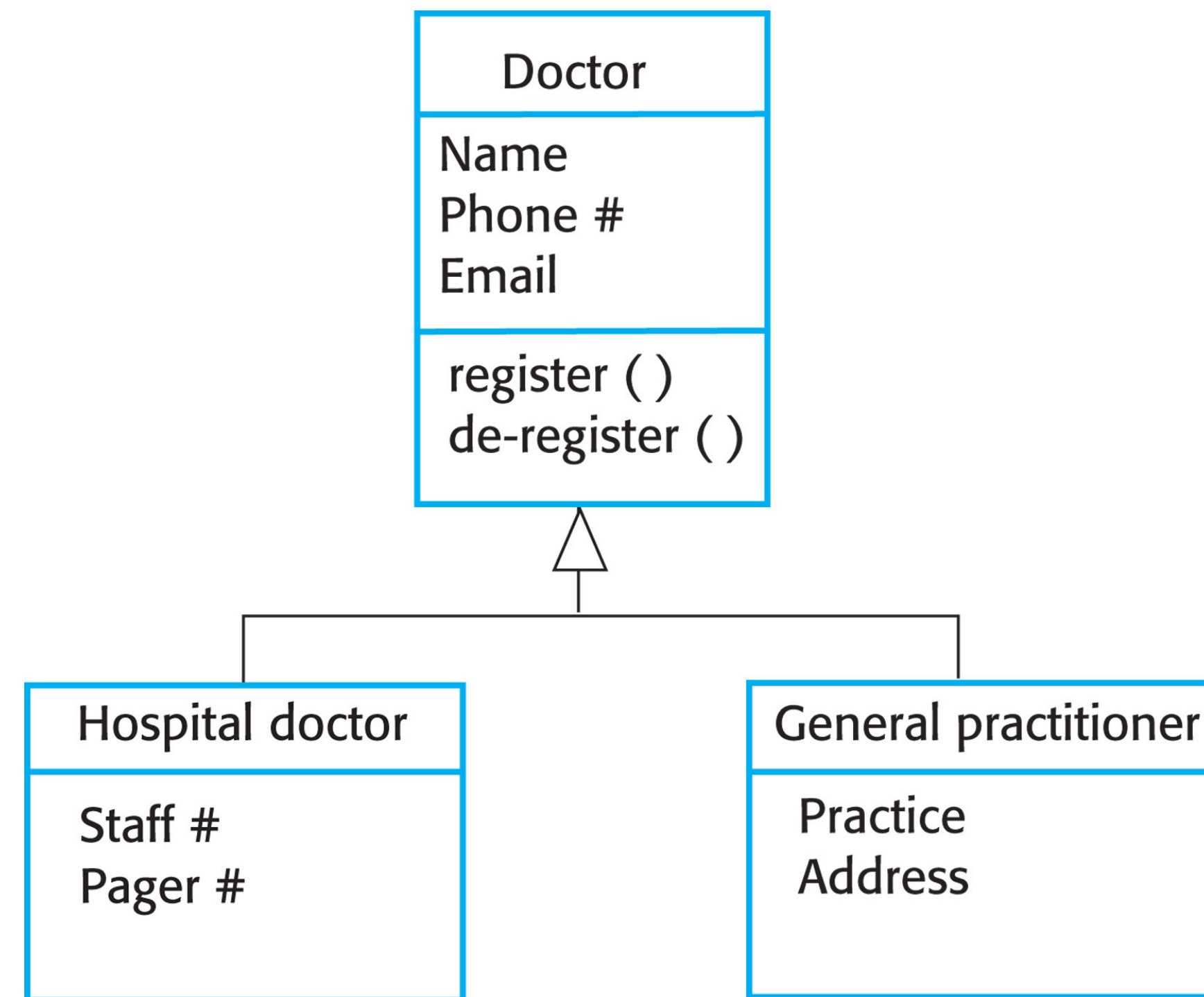
Figure 5.10 A Consultation class



Copyright ©2016 Pearson Education, All Rights Reserved



Figure 5.12 A generalization hierarchy with added detail



Copyright ©2016 Pearson Education, All Rights Reserved



Class Description Documents

- Adding detail to Classes
 - Both guides developers, and acts as v1 of documentation
- For each variable
 - what data format
 - what will it be used for
- For each method/function
 - what information does it receive (and why)
 - what does it do
 - what output does it produce (and why)
 - who will use the output
- And finally: how to know it works as expected

Consultation
Doctors Date Time Clinic Reason Medication prescribed Treatment prescribed Voice notes Transcript ...
New () Prescribe () RecordNotes () Transcribe () ...

Copyright ©2016 Pearson Education, All Rights Reserved



Handout 1



University of
Nottingham

UK | CHINA | MALAYSIA

Test Plans

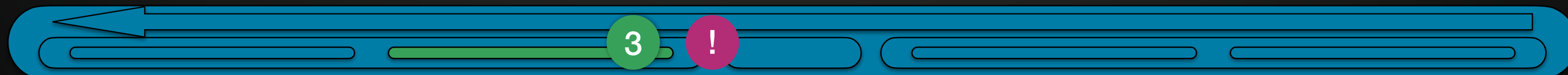


Test Plans

- Something often ignored by students **before they begin to code**
- Determines when a programmer has ‘finished’ building something
- Used for Test Driven Development (future lecture)

“Testing is designed to show that the software does what it is supposed to and discover program defects before it is put to use”

- They are, therefore, part of the design phase



Test Plans

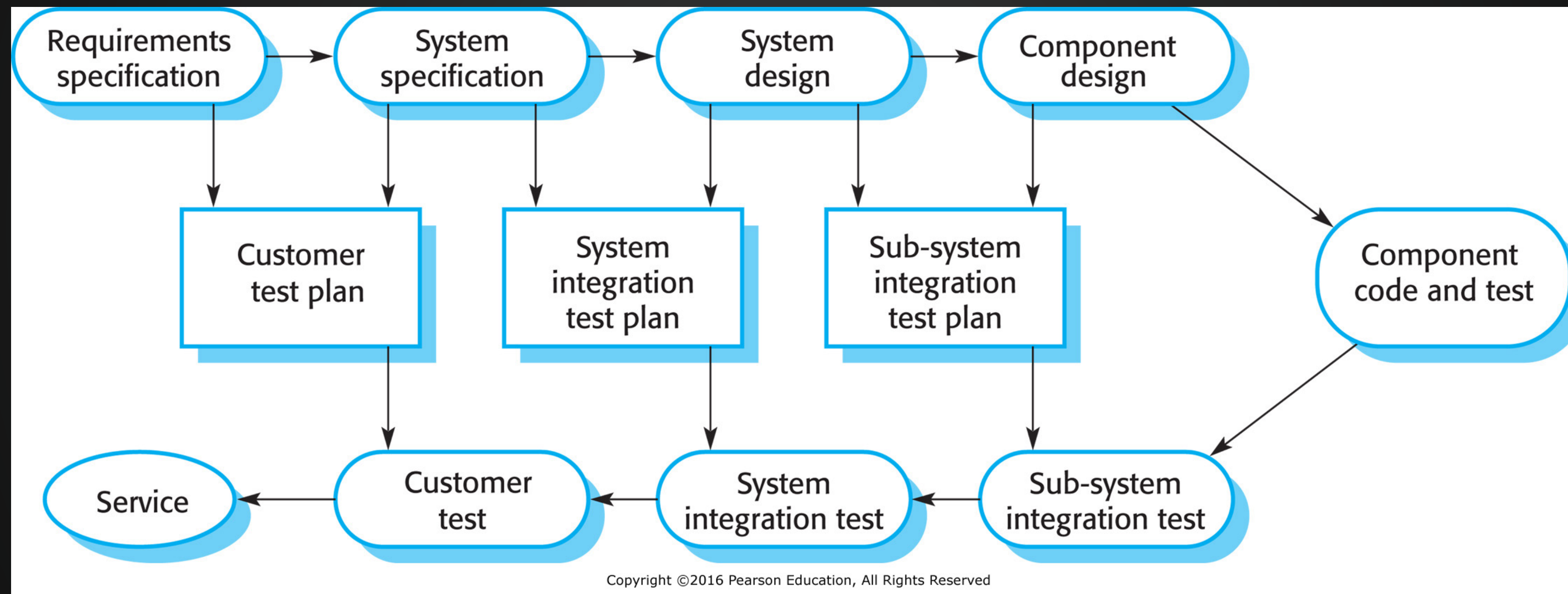
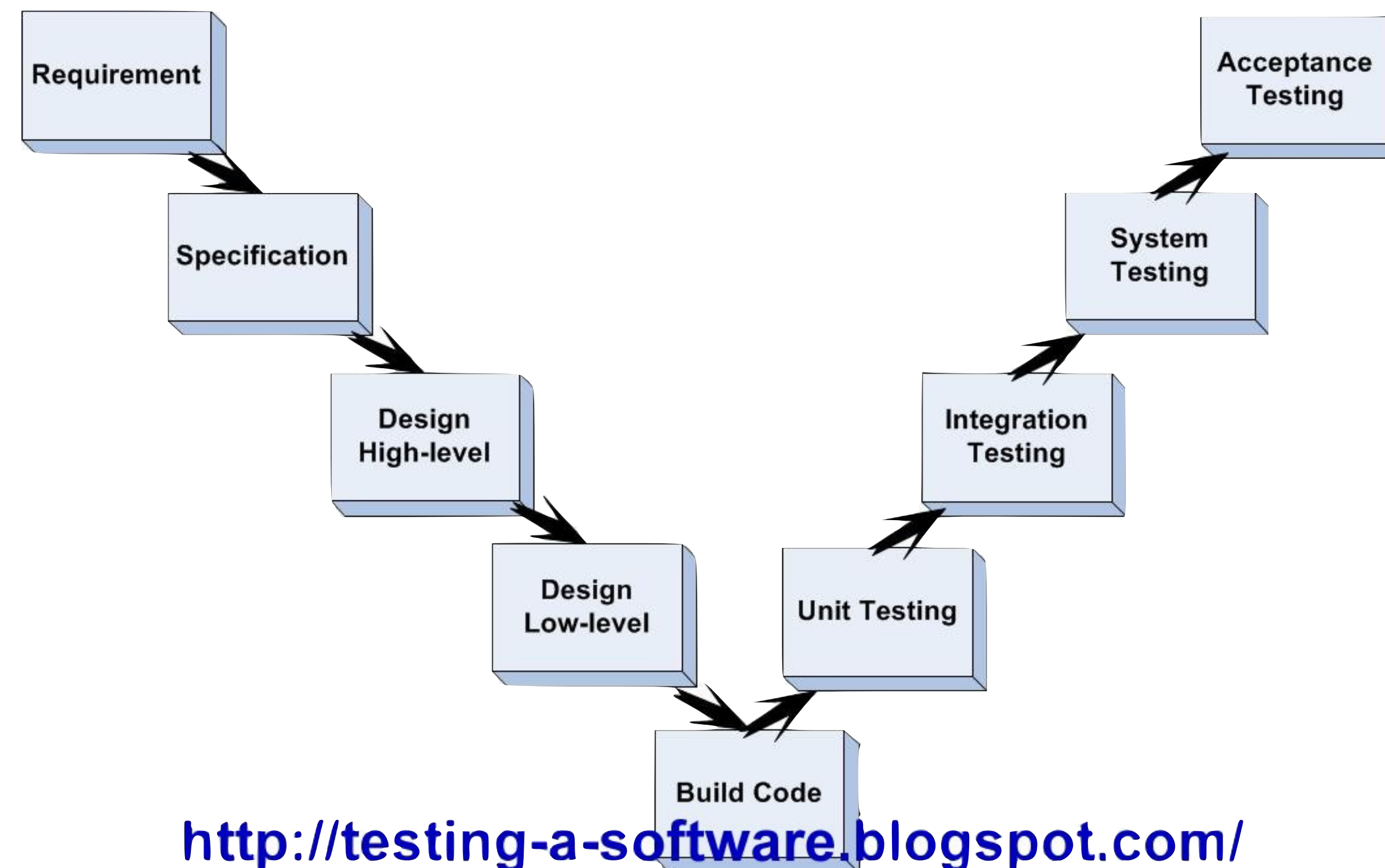


Figure 2.7 Testing phases in a plan-driven software process



Test Plans



Test Plans

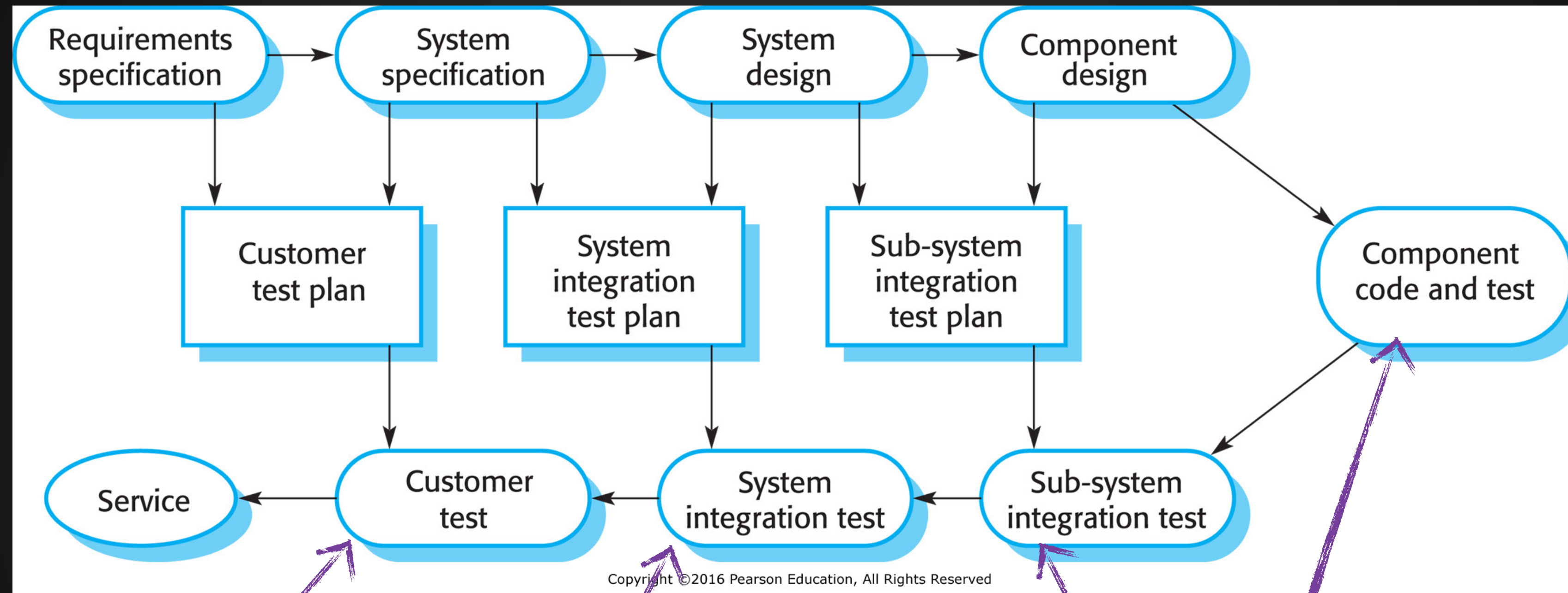


Figure 2.7 Testing phases in a plan-driven software process

Acceptance Testing

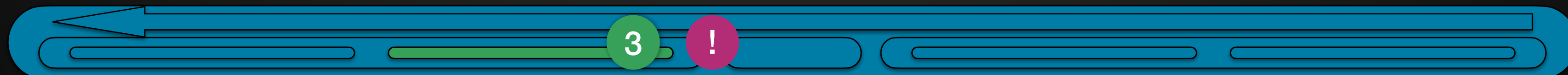
Release Testing

Development Testing



Test Plans

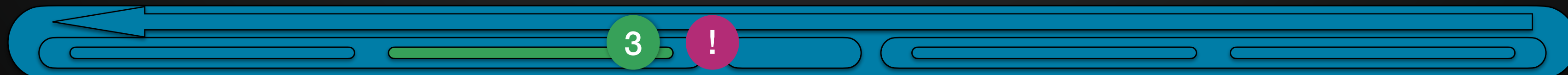
- **Development Test Plans**
 - lets us prove that a class functions correctly
 - who will do it, with what data, on which platform
- **System/Integration Test Plans**
 - lets us prove that the software meets the Specs
 - tests that cause one class to use another etc
 - or test that check that components interface correctly
- **Acceptance Test Plans**
 - shows that the software meets the requirements
 - often done with client - so they 'accept' the software





Test Plans - what used for?

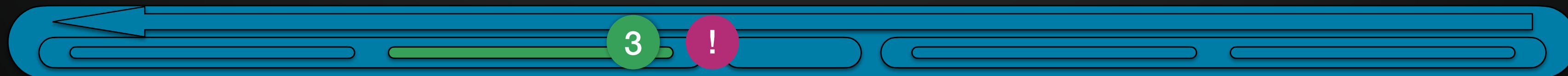
- To define what counts as 'finished'
 - and how each stage back up the V is tested
 - where when by who etc
- 1) Developers use TPs to test code before delivering it
- 2) Managers use TPs can estimate testing workload, and schedule it
 - and include it in the budget
- 3) Testing documentation serves as evidence to clients
 - of proper Software Engineering





Test Plans - The Overall Document

- Testing Process - description of approach taken
- Specifications Traceability - links between Specs and Tests
- Tested Items - list of things to be tested
- Testing Schedule - schedule in relation to overall project
- Test Recording Procedures - how test results will be recorded
- Hardware/Software Requirements - for testing machines
- Constraints - number of people, machines, etc needed
- **System Tests - a list of all the exact test cases that will be tested**



Test Plans - Designing Test Cases

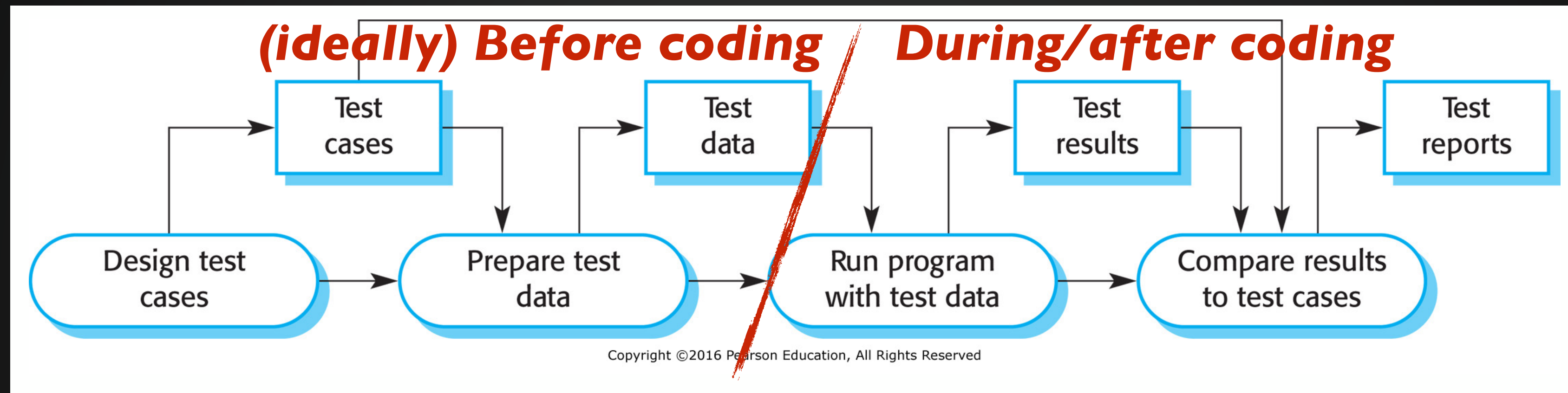


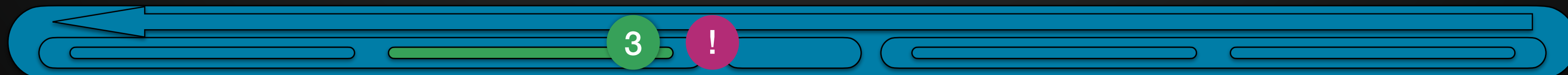
Figure 8.3 A model of the software testing process

- Now - you plan the test cases, and what data will test it
- After Built - you run the the test, until it passes



Test Plans - Each Test Case

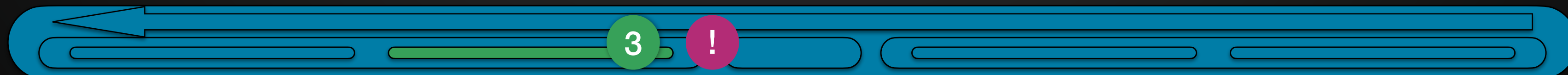
- 1) A statement of what is being tested
- 2) Specifications of the inputs to the test
- 3) What the expected output from the system is
- 4) Specify **the steps needed** to carry out the test
 - (if its not just 'run module with test data')
 - e.g. it might require other functions to be called first





Test Plans - Types

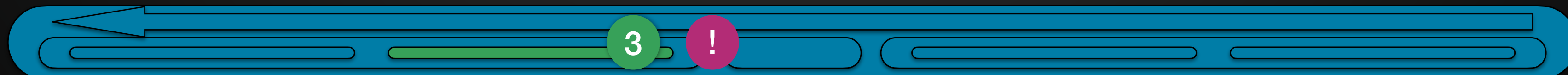
- Validation Testing
 - tests that show the software produces the right answer (when you give it all types of correct data)
- Defect Testing
 - tests that show the software doesn't break (when you give it all types of incorrect data)
- Writing tests for Validation only, is a **common mistake**





Test Plans » Test Documentation

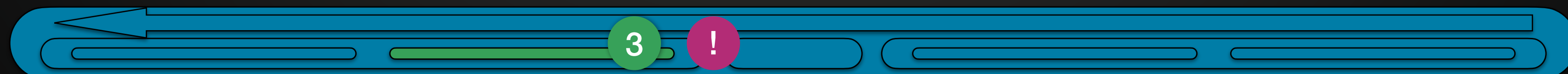
Test ID	Reason	Input	Expected Output	Pass/Fail
1	A description of what we are testing	-1	error	
2	A description of what we are testing	-1	error	
3	A description of what we are testing	-1	error	
4	A description of what we are testing	-1	error	





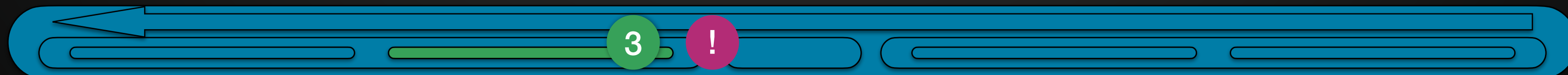
Test Plans » Test Documentation

Test ID	Reason	Input	Expected Output	Pass/Fail
1	A description of what we are testing	-1	error	
2		4.99	converted to 5	
3		1,000,000,000,000	error	
4	The next thing we are testing	-1	error	





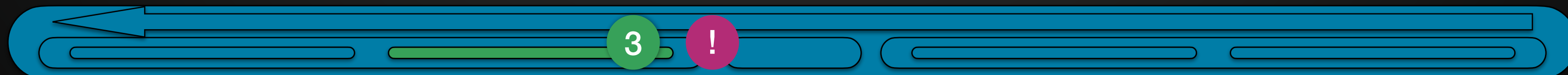
Activity





Test Plans » Test Documentation

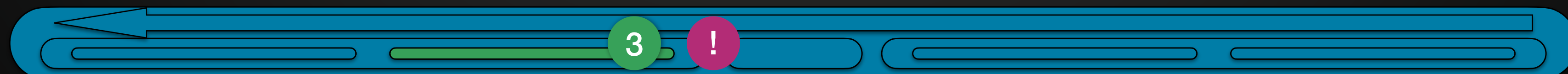
Test ID	Reason	Input	Expected Output	Pass/Fail
1	A description of what we are testing	-1	error	
2		4.99	converted to 5	
3		1,000,000,000,000	error	
4	The next thing we are testing	-1	error	





Test Plans » Test Documentation

Test ID	Reason	Input	Expected Output	Pass/Fail
1	A description of what we are testing	-1	error	Pass
2		4.99	converted to 5	Pass
3		1,000,000,000,000	error	FAIL
4	The next thing we are testing	-1	error	Pass





Think for a moment

- What happens when a test fails?
- You dont ship it with a fail...



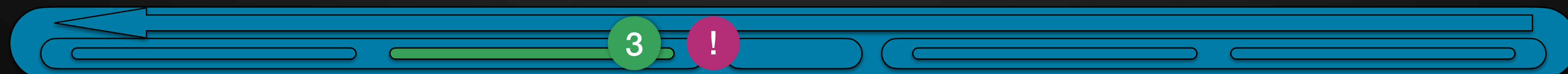
Links between Code and Test Reports

- If code changes because of a test, the code should cross-reference the test/result, and who made it, and when.
- These link together
- If you find a problem in the code with your tests, the report should identify what change was made (was the test wrong? Was the code wrong?). And then show that it passed after the change.



Test Plans » Test Documentation

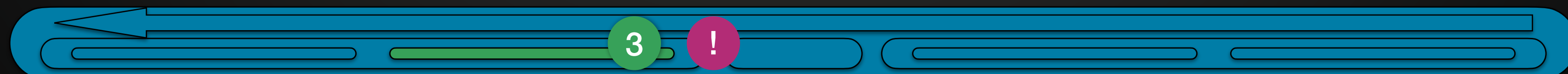
Test ID	Reason	Input	Expected Output	Pass/Fail
1	A description of what we are testing	-1	error	Pass
2		4.99	converted to 5	Pass
3		1,000,000,000,000	error	1) FAIL 2) Pass
4	The next thing we are testing	-1	error	Pass





Test Plans » Test Documentation

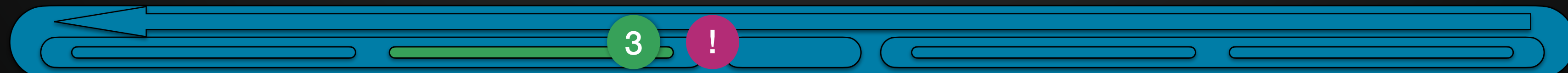
Test ID	Reason	Input	Expected Output	Pass/Fail
1	A description of what we are testing	-1	error	Pass (date)
2		4.99	converted to 5	Pass (date)
3		1,000,000,000,000	error	1) FAIL (date) 2) Pass (date)
4	The next thing we are testing	-1	error	Pass (date)





Test Plans » Test Documentation

Test ID	Reason	Input	Expected Output	Pass/Fail	Notes
1	A description of what we are testing	-1	error	Pass (date)	
2		4.99	converted to 5	Pass (date)	
3		1,000,000,000,000	error	1) FAIL (date) 2) Pass (date)	1) rounded to MAXINT ACTION: check for x>MAXINT
4	The next thing we are testing	-1	error	Pass (date)	





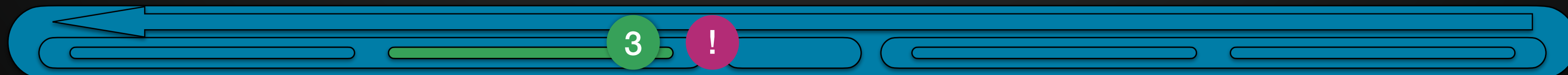
Overview

Chromium development is heavily test driven. In order to maintain a rapid rate of development across multiple platforms and an ever increasing set of features, it is imperative that test suites be updated, maintained, executed, and evolved. Any new features should have test coverage and in addition most changes should have test coverage. As a contributor to Chrome you are expected to write quality tests that **provide** ample code coverage. As a reviewer you are expected to ask for tests. **The [Chromium Continuous Integration system](#) is employed to run these tests 24x7.**



Higher-Level Test Documents

- Would look the same in principle
- but the inputs, outputs etc are different
 - would be e.g. Requirements Use Cases for Acceptance Testing
 - or e.g. Specifications for System Testing
 - rather than inputs for function calls, and expected return values
- You can still document pass/fail/dates/reasons





Today's Learning Outcomes

- 0) Low-level designs are there to guide developers
- 1) These are the final outputs of the Spec phase
- 2) Planning testing is part specifying phase

